

CS1CA Computer System Architecture
Computer performance measures
Week 4 lecture 2 of autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher

Preamble: materials delivered in weeks 4(2)-12

- RISC (Reduced Instruction Set Computer) is a type of microprocessor architecture popularly used in modern computers, especially in mobile devices, due to its simplified set of instructions, hence lower power-consumption.
- Throughout weeks 4(2)-12 , we use **DLX** architecture, an educational edition of RISC architecture, in delivering topics. Further readings refer to the recommended Patterson & Hennessy books.
- The following topics will be delivered in these weeks.
 - Computer performance measures
 - Instructions: machine language
 - Processor and pipelining
 - Memory hierarchy
 - Storage and I/O systems



In this lecture

- Contents to be delivered
 - Computer performance measures: execution time, CPU time, and Amdahl's law
- Learning outcomes are listed below.
 - To be able to use benchmark programs to measure computer performance and to report performance results
 - To be able to measure computer performance by using CPU time
 - To be able to articulate Amdahl's law as a quantitative approach in guiding computer system design to gain the overall “speedup” (performance improvement).

Aims of computer system design

- Computer architecture is concerned with computer system design.
- It mainly refers to three aspects of computer design.
 - instruction set architecture;
 - organisation: memory system, bus structure, internal CPU;
 - hardware: detailed logic design and packaging technology.
- Computer architecture aims to improve computer **performance** and to reduce cost of computer systems.

Computer Performance

- Measuring performance

Execution time: the time between the start and the completion of an event.

$$performance = \frac{1}{execution\ time}$$

A program is executed in two computers A&B.
Computer A: ET=20sec, performance A=1/20
Computer B: ET=60sec, performance B=1/60

(Performance A)/(performance B) =
(1/20)/(1/60) = 3

Computer A is 3 times faster than Computer B.

Execution time includes

CPU time: time spent on computation – including user CPU time and system CPU time

Elapsed time : time waiting for I/O or running other programs

Programs used for performance measurement: benchmark

- Real programs
 - compilers, any application programs.
- Kernels
 - key portions extracted from real programs.
- Benchmark suites
 - benchmark is typical 10-100 line programs.
 - benchmark suites: collection of benchmark programs.

Two conventional methods to calculate execution time for multi-testing programs

- Average Execution Time T_a

: execution time for the i th program

- Weighted Average Execution Time T_a

, and $\sum w_i = 1$

: weight for the i th program

In both cases, n is the number of programs involved.

Reporting performance

In reporting the performance measurement result, along with the execution time, the following items should be included.

Hardware	Software
Model number	O/S and version
CPU capacity	Compilers and version
Number of Caches	File system type
Memory capacity	System state
Disk subsystem	

CPU performance

- CPU time spent for running a program is used to measure CPU performance.

CPU time = CPU clock cycles for a program × clock cycle time

or
$$CPU\ time = \frac{CPU\ clock\ cycles\ for\ a\ program}{clock\ rate}$$

- Decompose the CPU clock cycles into IC and CPI.

- **IC**: Instruction Count

- **CPI**: clock Cycles used by Per Instruction

$$CPI = \frac{CPU\ clock\ cycles\ for\ a\ program}{IC}$$

- **Clock cycle time**: time of one clock cycle

$$clock\ rate = \frac{1}{clock\ cycle\ time}$$

Equations for calculating CPU time

$$CPU\ time = \frac{IC \times CPI}{clock\ rate} \quad \text{or}$$

$$CPU\ time = IC \times CPI \times clock\ cycle\ time$$

CPU performance is dependent on:

- **Clock rate (or clock cycle time)**

Hardware technology and organisation

- **CPI**

Organisation and instruction set architecture

- **IC**

Instruction set architecture and compiler technology

$$\begin{aligned} \text{CPU Performance} \\ = 1 / (\text{CPU time}) \end{aligned}$$

When various types of instructions run in a CPU

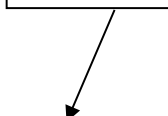
- Total CPU clock cycles

$$CPU \text{ clock cycles} = \sum_{i=1}^n CPI_i \times IC_i$$

- Average CPI

$$CPI = \frac{\sum_{i=1}^n CPI_i \times IC_i}{instruction\ n\ count} = \sum_{i=1}^n CPI_i \times \left(\frac{IC_i}{instruction\ n\ count} \right)$$

Frequency: percentage
of i type instructions



- CPU time

$$CPU \text{ time} = \left(\sum_{i=1}^n CPI_i \times IC_i \right) \times Clock \text{ cycle time}$$

$i = 1, 2, \dots, n$ representing no. of instruction types

Quantitative principles of computer design:

Amdahl's Law

- **Amdahl's Law** is a fundamental law used to quantify performance improvements in computer design.
- Parameter of *speedup* is used for representing the improvement quantitatively.

Quantitative principles of computer design: definition of *Speedup*

$$\textit{speedup} = \frac{\textit{performance with enhacement}}{\textit{performance without enhacement}}$$

or

$$\textit{speedup} = \frac{\textit{execution time without enhacement}}{\textit{execution time with enhacement}}$$

$$\textit{speedup} \geq 1$$

There are two factors involved in Amdahl's Law, **FE** and **SE**.

- **FE (Fraction Enhancement)**: fraction of the computation time in the original machine that can be converted to take advantage of the enhancement.

$$FE \leq 1$$

Example:

An original program takes 100 secs as execution time, and within the total 100 secs, only 40 secs of the program can be enhanced. Therefore, $FE = 40/100 = 0.4$

- **SE (Speedup enhancement)**: how much faster the program would run if the enhanced mode was used for the entire program

$$SE \geq 1$$

Example:

If an enhanced mode takes 4 secs for a program that requires 40 secs without the enhanced mode, then $SE = 40/4=10$

- For a program, some portion of it has been enhanced, and some portion of it remains unenhanced.
- The new execution time for the program is the sum of the time spent by the unenhanced portion and the time spent by the enhanced portion.
- Amdahl's law can be written as

$$execution\ time_{new} = execution\ time_{old} \times \left[(1 - FE) + \frac{FE}{SE} \right]$$

$$speedup_{overall} = \frac{execution\ time_{old}}{execution\ time_{new}} = \frac{1}{(1 - FE) + \frac{FE}{SE}}$$

Derive it based on the definition!

Example

- A CPU is usable 0.4 of the total execution time;
- The enhanced CPU runs 10 times faster;
- What is the overall speedup of the new machine?

$$FE = 0.4, SE = 10$$

$$speedup_{overall} = \frac{execution\ time_{old}}{execution\ time_{new}} = \frac{1}{(1 - 0.4) + \frac{0.4}{10}}$$

$$speedup_{overall} = 1.56$$

End of this lecture

Questions and exercises (1)

- How is computer performance measured?
- How is the CPU performance measured?
- What types of code can be used as benchmark programs in computer performance measurement?
- Why is Amdahl's law important in computer design?
- What is the relationship between CPU performance and the overall system performance? Does Amdahl's law give you a hint about it?

End of this lecture

Questions and exercises (2)

- A benchmark program is used for measuring CPU time of a computer system. There are 3 types of instructions involved in the program. After compiling, the following information is recorded.

IC1	CPI1	IC2	CPI2	IC3	CPI3
20	5	30	8	50	2

Calculate the average CPI.

If the CPU clock rate is 3GHz, what is the CPU time of running the program?

Hint: refer to equations on slide 11.

End of this lecture

Questions and exercises (3)

- To improve a computer system's performance, a new design aims to have the overall system speedup to 1.5 times of the original version. It is known that the system performance is contributed by both CPU (40%) and memory (60%). In the new design, only the CPU time can be changed.

By using the Amdahl's law, calculate how much faster the new CPU should be to meet the requirement of the overall speedup.